Lesson notes · for you

Timetables with CP-SAT

Optimization · Level 3 · Lesson 4

Time
20 min

Materials
Paper and a pen; A list of clubs, tasks or subjects and who takes part in each; Optional: a phone with Darsi to run your model

Learning goals

- Model a club timetable in CP-SAT with all-different rules.
- Explain propagation and search, and what INFEASIBLE proves.
- Add a fairness rule and judge the result.

Activity: Model a real week

1. List (5 min): Write four to six tasks (clubs, revision subjects or chores), the days available, who is involved in each, and every rule you know.
2. Model (8 min): Write one day variable per task, one all-different line for each person's tasks, and one line for each fixed or forbidden day.
3. Run (4 min): Run the model in the code step or the modeller lab. Read the status first: OPTIMAL, or INFEASIBLE and why.
4. Fairness (3 min): Find who gets the worst slot. Add one rule that shares it and run again. Did anything else have to move?

No phone? Do it on paper: cross out impossible days for each task, then guess, as the solver does.

Discussion

- Why doesn't the solver need to try every plan?
- What should you do when the answer is INFEASIBLE?
- Is a timetable that keeps every rule always fair?

Check yourself

- One rule I would write as a line + one fairness rule for my week.

Try it this week
Model a week of your own (your study plan, or your family's chores) with the same few lines: one variable per task for its day, a rule for anything that can't share a day, and any fixed days. Run it, look at who gets the worst slot, then add a rule to share it out and run it again.

Worksheet

Timetables with CP-SAT

Optimization · Level 3 · Lesson 4

Name _____ Class _____ Date _____

1 **Your club model says INFEASIBLE. What should you do first?**

- ◯ Run it again: it might work next time
- ◯ Delete rules until it works
- ◯ Find which rules clash, and decide with people which can bend

2 **Which of these can you write as a rule (a constraint) in the club model? Choose all that apply.**

- ☐ Coding and football meet on the same day
- ☐ The timetable everyone likes best
- ☐ Coding meets before drama
- ☐ Sara's clubs are on different days
- ☐ Football is not on day 3

3 **With four clubs and three afternoons there are 81 possible plans, so the solver checks all 81.**

- ◯ True
- ◯ False

4 **Four clubs meet on days 1 to 3, and the football field is busy on day 3. Each list in pupils is one student's clubs. Add the rule that each student's clubs are all on different days, then run it.**

```
from ortools.sat.python import cp_model
clubs = ["chess", "football",
         "drama", "coding"]
pupils = [["chess", "drama", "coding"],
          ["football", "chess"],
          ["drama", "football"]]
m = cp_model.CpModel()
day = {c: m.new_int_var(1, 3, c)
       for c in clubs}
# each pupil: their clubs on different days

m.add(day["football"] <= 2)
solver = cp_model.CpSolver()
print(solver.status_name(solver.solve(m)))
```

Write your program:

5 **Complete the end of the program: solve the model, and if it found a timetable, print each club's day.**

```
solver = cp_model.CpSolver()
status = solver._____(m)
if status == cp_model._____ or status == cp_model.FEASIBLE:
    for c in clubs:
        print(c, solver._____(day[c]))
else:
    print("No timetable:", solver.status_name(status))
```

Word bank: `OPTIMAL` `solve` `add` `INFEASIBLE` `maximize` `value`

✎ **Which timetable in your life (school, clubs, chores) puts the same people in the worst slot every time? How would you write a fairer rule?**

Check your answers

Timetables with CP-SAT

Optimization · Level 3 · Lesson 4

1 **Find which rules clash, and decide with people which can bend**

INFEASIBLE is the solver's proof that no timetable keeps all the rules, so running it again gives the same answer.

2 **Sara's clubs are on different days; Football is not on day 3; Coding meets before drama; Coding and football meet on the same day**

Each is one line: add_all_different for Sara, day["football"] <= 2, day["coding"] < day["drama"], and day["coding"] == day["football"].

3 **False**

The solver crosses out impossible days first and guesses only when stuck, so it usually looks at a tiny part of the plans.

4

```
from ortools.sat.python import cp_model
clubs = ["chess", "football",
         "drama", "coding"]
pupils = [["chess", "drama", "coding"],
          ["football", "chess"],
          ["drama", "football"]]
m = cp_model.CpModel()
day = {c: m.new_int_var(1, 3, c)
       for c in clubs}
for p in pupils:
    m.add_all_different([day[c] for c in p])
m.add(day["football"] <= 2)
solver = cp_model.CpSolver()
print(solver.status_name(solver.solve(m)))
```

One all-different rule per student keeps their clubs apart.

5

```
solver = cp_model.CpSolver()
status = solver.solve(m)
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    for c in clubs:
        print(c, solver.value(day[c]))
else:
    print("No timetable:", solver.status_name(status))
```

solve runs the search and returns a status.